

Spec-Driven-Development in der Praxis

Wie wir Spec-Driven Development bei eventim leben

Fabian Meyer
Chapter Lead **Vue** @ eventim Tech

whoami

Chapter Lead **Vue** @ eventim Tech

- Leitung unseres Vue Chapters - Standards, Tooling, Developer Experience
- Tech Lead für den eventim.Tixx Ticket-Onlineshop
- Leidenschaft für Agentic Engineering und alle anderen KI-Themen

Privat: Gym, Laufen, 3D-Druck und viel zu viele Homeserver-Projekte



eventim & Tixx

eventim ist Europas führender Ticketing-Anbieter.

eventim.Tixx ist unsere Ticketing-Plattform für Sport & Festivals: 200+ Kunden, PHP & Vue Frontends, Legacy-Core, Multi-Team GitLab Setup.

Der Anfang mit KI

Die ChatGPT-Aera

- Gespräche mit Datenschutz und IT-Security
- ChatGPT als Ersatz/Begleiter für Stack Overflow und Google
- Code-Snippets kopieren, selbst zusammenkleben

Dazwischen kam Copilot - Autocomplete auf Steroiden.

Codex & Claude Code

- Code-bewusste Tools mit echtem Kontext
- Plan-Modus → Implementierung → Coding mit Inferenz-Geschwindigkeit
- Bugs gefixt, kleine Features gebaut, Refactorings, Dokumentation

METR-Studie: Erfahrene Devs anfangs 19% langsamer mit KI. Nach Einarbeitung: 18% schneller.

Schnellerer Code, geringer Impact - was, wenn Coding nie der Flaschenhals war?

Der Agent schreibt Code. Du machst alles andere.

- ✕ Du liest das Jira-Ticket und übersetzt es in einen Plan
- ✕ Du lässt Claude den Plan umsetzen und testest das Ergebnis selbst
- ✕ Du erstellst einen Merge Request und weist Reviewer zu
- ✕ Du babysittest die Pipeline: Linting, Code-Style, Unit Tests, Security Scans
- ✕ Du musst auf einer Dev/Stg-Instanz deployen für QA
- ✕ Ein E2E-Test schlägt fehl
- ✕ ... und so viel mehr

Code schreiben war nie der Flaschenhals.

In Enterprise ist der Prozess *rund um* den Code, wo man stecken bleibt.

Code wird schnell generiert. Durch den Prozess kommen? Immer noch langsam.

"Dann werfen wir den Prozess halt weg, oder?"

Das geht nicht - und das wollen wir auch nicht.

- Human-in-the-Loop für Code Review und QA
- Standards, Guardrails und Security
- Komplexe Infrastruktur, Multi-Repo, etablierte Prozesse

Wir wollen Automatisierung - aber wie kommen wir dahin?

Was ist Spec-Driven Development?

Nur ein Buzzword? Jein. In der Essenz sind es 3 Schritte:



Spec

Einen detaillierten Plan mit dem Agenten erstellen, in dem **du** die Entscheidungen triffst.



Code

Den Agenten in einer **neuen Session** implementieren lassen - basierend auf der Spec.



Verify

Den Agenten den Code **gegen mehrere Quality-Gates** verifizieren lassen.



Warum bringt das so viel? Stichwort Kontext. Je voller das Context Window, desto schlechter der Output. Die Spec hält den Kontext fokussiert.

```
/context
Context Usage
[Progress bars] Sonnet 4.6
[Progress bars] eu.anthropic.claude-sonnet-4-6
[Progress bars] 14.5k/200k tokens (7%)

Estimated usage by category
[System prompt]: 3.3k tokens (1.7%)
[System tools]: 6.7k tokens (3.3%)
[Custom agents]: 464 tokens (0.2%)
[Memory files]: 967 tokens (0.5%)
[Skills]: 3k tokens (1.5%)
[Messages]: 119 tokens (0.1%)
[Free space]: 152.5k (76.2%)
[Autocompact buffer]: 33k tokens (16.5%)
```

SDD-Frameworks im Vergleich

Framework	Schritte	Stärke
OpenSpec	Discover → Specify → Implement → Validate	Formale Spezifikation, maschinenlesbar
BMAD	Brainstorm → Map → Architect → Develop	Kreative Exploration, Architecture-First
PlanAct	Plan → Act → Reflect → Iterate	Minimalistisch, universell anwendbar
SpecKit	Spec → Build → Test → Ship	GitHub-native, opinionated Templates

Was fällt auf?

- Alle generieren eine Art **Spezifikation** oder Plan
- Alle lassen den Agenten **implementieren**
- Die meisten haben eine **Verifikation** am Ende

Alle machen Spec → Code → Verify. Nur die Schritte dazwischen variieren.

Sie sind ein guter Startpunkt. Ausprobieren, anpassen, nutzen. Da steckt keine Magie drin.

Was für uns nicht passte

- ✕ Die meisten Frameworks gehen von **GitHub** aus - wir nutzen GitLab
- ✕ Sie nehmen **einfache Deploys** an - wir haben komplexe Multi-App-Infrastruktur
- ✕ Sie erwarten ein **einzelnes Repo** - wir arbeiten mit mehreren Repos
- ✕ Sie setzen einen **flexiblen Prozess** voraus - wir haben einen etablierten Enterprise-Prozess

Wir wollten unseren Prozess nicht aufgeben - wir wollten ihn (teil)automatisieren.

Wir wollten unser eigenes SDD-Framework bauen, damit wir auch verstehen womit wir arbeiten.

Den Agenten "einarbeiten"

Welche Infos braucht unser Agent um produktiv zu sein?



Jira-Zugang

Tickets lesen, kommentieren, verschieben



GitLab-Zugang

Branches, MRs, Pipelines, Code Review



Deploy-Fähigkeit

Komplexe Multi-App-Deployments ausführen



Pipeline-Monitoring

Pipelines beobachten, auf Fehler reagieren



Dokumentation

So dokumentieren wie wir es tun



Human-in-the-Loop

Pairing = Mensch im Loop

Das klingt wie einen neuen Kollegen einarbeiten. Genau das ist es.

Integration von Tools und Prozess



Skills vs. MCP

- **Skills:** Markdown-Dateien mit Anweisungen. Ideal für Team-Workflows und Prozesse. Leicht zu reviewen.
- **MCP:** Server-basierte Tool-Integration. Ideal für externe APIs und Datenquellen.
- Wir nutzen beides - Skills für Prozess, MCP für Tool-Anbindung wo nötig.



Unseren Prozess abbilden

- Wie lesen wir Jira-Tickets? Welche Details stehen drin?
- Wie dokumentieren wir? Welche Guidelines und Guardrails haben wir?
- Welche Commands muss der Agent ausführen können?
- Das gesamte Developer-Setup: lokal bevorzugt, weil schneller zum Iterieren.

Lokale Ausführung > Remote-Ausführung. Schneller, einfacher zu iterieren.

Unser SDD-Framework

Spec-Driven Development bei eventim. Drei Phasen - gehen wir ins Detail.



Spec

Jira-Ticket holen,
Codebase recherchieren,
gemeinsam spezifizieren.
Output: strukturierte Spec
als Kontext-Artefakt.



Code

Neue Session, TDD-Zyklus,
Conventional Commits,
Push zum Remote. Agent
arbeitet autonom bis alle
Tasks erledigt sind.



Verify

Spec-Abgleich, lokale
Checks, Code Review
durch frischen Agenten,
Dev-Test mit Playwright.

Drei Phasen. Gehen wir ins Detail.

Spec: Kontext sammeln

 **Was wollen wir tun?** Kontext aus Jira: Ticket, Akzeptanzkriterien, Beschreibung, verlinkte Tickets.

 **Womit arbeiten wir?** Codebase aus Git: verwandte Komponenten, Patterns, Architektur.

 **Brainstorming & Q&A:** Agent erstellt Empfehlungen, geht in Q&A-Session. Gemeinsames Verständnis erarbeiten.

 **Output: Strukturierte Spec.** Recherche, Einschränkungen, Anforderungen, Entscheidungen, Tasks, Verifikationskriterien.

Die Spec ist der Kontext-Transfer. Ohne sie müsste die nächste Session alles nochmal herleiten.

Spec in Aktion

- Agent holt das **Jira-Ticket** und liest Akzeptanzkriterien
- Erkundet die Codebase nach verwandten Komponenten
- Interaktives Q&A: Klärungsfragen, Optionen vorschlagen
- Schreibt strukturierte Spezifikation
- Zusammenfassung als Jira-Kommentar



```
Claude Code
Opus 4.6

> /g-plan TIXX-4821

TIXX-4821 aus Jira laden ...
"Event-Filter-Komponente hinzufügen"
Akzeptanz: Datum-, Kategorie-, Ort-Filter

• Read src/components/EventList.vue
• Read src/components/FilterBar.vue
  └─ 2 verwandte Komponenten gefunden

? Soll der Filter-State in der URL persistiert werden?
  → ja, als Query-Parameter

? Edge Case - was wenn keine Ergebnisse?
  → Empty State mit Reset-Link anzeigen

Spezifikation wird geschrieben ...
✓ specs/TIXX-4821_event_filter.md bereit
✓ Zusammenfassung an Jira gepostet
```

Die Spec wird zum Vertrag zwischen dir und dem Agenten.

Beispiel-Spec

```
---  
id: TIXX-4821  
title: Event-Filter-Komponente hinzufügen  
status: draft  
branch: g/TIXX-4821_event-filter-komponente  
repos: [ols]  
---  
  
## Context  
  
Der Onlineshop zeigt Events ohne Filtermöglichkeit an.
```

Code: Implementieren

📄 **Neue Session, Spec als Kontext.** Der Agent liest die Spec und hat damit alles was er braucht.

🔧 **TDD ist jetzt noch wichtiger.** RED → GREEN gibt dem Agenten einen eingebauten Verifikations-Loop. Gute Task-Liste, die der Agent abarbeiten kann.

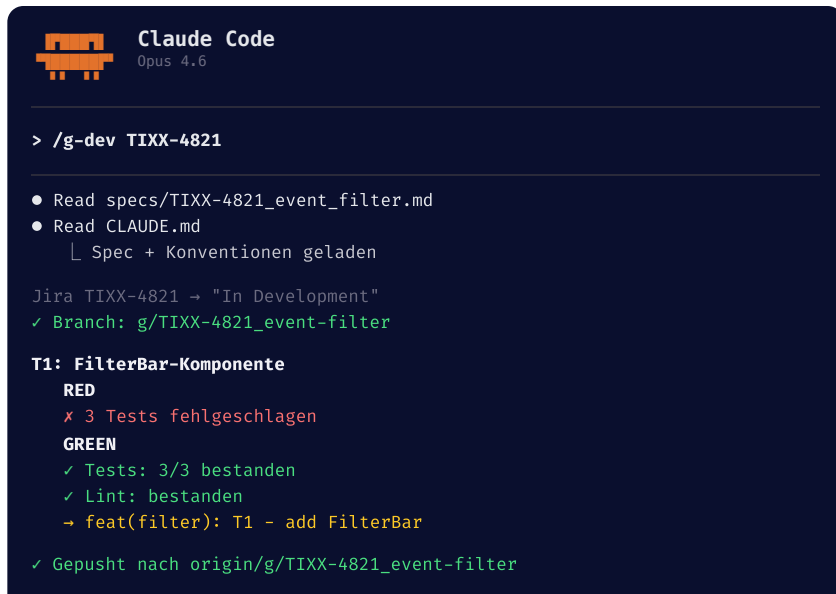
🔗 **Conventional Commits, kleine logische Einheiten.** Oft committen, klare Commit-Messages. Einfach für den Agenten, gut für das Team.

📦 **Push zum Remote.** Der Agent pusht seinen Stand um einen ersten "Done"-State zu erreichen.

Der einfachste Schritt - weil die Spec die schwere Arbeit schon erledigt hat.

Code in Aktion

- Agent liest die Spec und lädt Projekt-Konventionen
- Verschiebt Ticket nach "In Development", erstellt Branch
- Implementiert Task für Task im TDD-Zyklus
- **RED** → **GREEN** → Lint → Commit
- Pusht zum Remote



```
Claude Code
Opus 4.6

> /g-dev TIXX-4821

• Read specs/TIXX-4821_event_filter.md
• Read CLAUDE.md
  └─ Spec + Konventionen geladen

Jira TIXX-4821 → "In Development"
✓ Branch: g/TIXX-4821_event-filter

T1: FilterBar-Komponente
RED
x 3 Tests fehlgeschlagen
GREEN
✓ Tests: 3/3 bestanden
✓ Lint: bestanden
→ feat(filter): T1 - add FilterBar

✓ Gepusht nach origin/g/TIXX-4821_event-filter
```

Keine Interaktion nötig bis alle Tasks fertig - Iterieren, committen, wiederholen.

Verify: Der schwierigste Schritt

-  **1. Spec-Abgleich.** Entspricht der neue Code der Spec? Intent, Anforderungen, Akzeptanzkriterien prüfen.
-  **2. Lokale Checks.** Lint, Tests, Type-Check, Kompilieren - alles was lokal geht. Auto-Fix bei Fehlern.
-  **3. Code Review.** Frischer Agent, klarer Kontext. Experimentiert: mit oder ohne Spec-Zugang. Auto-Fix der Findings.
-  **4. Dev-Test.** Der härteste Teil: echte User-Tests im Browser, Simulator oder CLI mit Playwright.

Jede Schicht hat einen Auto-Fix-Loop. Fehler werden sofort behoben und erneut geprüft.

Verify in Aktion

- Draft-Merge-Request mit **glab** erstellen
- Spec-Abgleich: alle Tasks erledigt, alle Anforderungen erfüllt
- Lokale Checks: Lint, Tests, Build
- Self-Review durch frischen Agenten
- Pipeline überwachen, bei Fehler fixen
- MR finalisieren, Ticket nach "Ready for Review"



Claude Code
 Opus 4.6

```

> /g-verify TIXX-4821

```

- Bash glab mr create --draft --title "TIXX-4821: Event filter"
 - └ MR !312 geöffnet (Draft)

Vollständigkeit: 4/4 Tasks ✓
 Anforderungen: 3/3 Must-Haves ERFÜLLT ✓

Lokal: lint ✓ | unit ✓ | build ✓

Self-Review (frischer Agent):

- "Fehlende Null-Prüfung in Zeile 47"

Fixing... gepusht

✓ Pipeline: grün

- Bash glab mr update 312 --ready
 - └ MR !312 bereit für Review
 - └ TIXX-4821 → "Ready for Review"

Bereit für das menschliche Review.

Dev-Test: Der härteste Teil

- Agent deployt auf Dev/Staging mit **/tixx-deploy**
- Oeffnet einen echten Browser mit **playwright-cli**
- Navigiert zum Feature, interagiert, verifiziert Verhalten
- Screenshots, Szenarien und Ergebnisse dokumentieren
- Bei Fehler: Code fixen, neu deployen, erneut testen



Claude Code
Opus 4.6

```

> /g-qa TIXX-4821

```

```

• Skill tixx-deploy --env staging --ticket TIXX-4821
✓ Deploy abgeschlossen
✓ Health Check: 200 OK

Playwright: Browser wird geöffnet...
✓ Navigiere → /events
✓ Datum-Filter anwenden → "März 2026"
✓ 12 Ergebnisse angezeigt
✓ Klick → "Rock am Ring 2026"
✓ Detail-Seite geladen

✓ Alle Szenarien bestanden
  | 6 Szenarien, 3 Screenshots dokumentiert
  | TIXX-4821 → "Ready for QA"

```

Der Mensch macht die finale QA - aber der Agent hat die Vorarbeit geleistet.

Unsere Erfahrungen

-  **Start:** Mit Mentoring angefangen. Daraus ein ganzes Framework entwickelt.
-  **Over-Engineering:** Zu rigide. Jeder Dev hat Präferenzen. Starres Framework passte nicht für alle.
-  **Lösung:** Shared Skill Set. Kern-Skills für alle + Freiheit für den individuellen Workflow.
-  **Tempo:** Alles entwickelt sich extrem schnell. Regelmäßiger Austausch essenziell. Vertrauen schwer aufzubauen.
-  **Ownership:** Du besitzt den Output. Bugs, Architektur und Security bleiben deine Verantwortung.

Perfekt ist der Feind von gut genug. Liefern, lernen, iterieren.

Was hat geholfen?

Weniger ist mehr.

 **Kurze, fokussierte Skills** — lieber klein und klar

 **Viele Tests (TDD)** — Verifikationsschicht für den Agenten

 **DDD** — kleine Kontexte, klare Grenzen

 **Coding Guidelines** — klare Regeln für Mensch und Agent

 **ADRs** — Agent versteht vergangene Entscheidungen

 **Saubere Architektur** — je klarer, desto besser navigiert der Agent

Nichts davon ist neu - aber alles davon ist jetzt wichtiger.

Einfluss auf die Zusammenarbeit

 **Product Manager:** Wir waren schnell unterwegs - er musste mithalten. Haben ihn mit allen Repos und eigenen Jira Skills aufgesetzt. Er hat jetzt technisches Wissen durch Claude.

 **Security-Team:** Gemeinsam Skills für Security-Checks gebaut. Agent kennt jetzt unsere Security-Anforderungen.

 **QA & UX:** Wir haben die Code-Repos für sie ausgecheckt, jetzt können sie auch Mockups direkt in der Applikation machen oder Elemente aus dem Code in Designs holen.

 **Überlappung:** PM gewinnt technisches Wissen. Wir treffen mehr Produkt-Entscheidungen. Rollen überlappen stärker - weil wir weniger refinieren und mehr nach Gefühl arbeiten.

Zum Experten gehen, gemeinsam einen Skill bauen - Inselwissen verschwindet.

Wo anfangen?

- 1 **Leichte Spec-Skill + Dev-Skill** — Starte mit einem einfachen Skill der eine Spec erstellt und einem der implementiert.
- 2 **Noch einfacher: Plan Mode** — Aber sage ihm er soll dich bis ins letzte Detail interviewen. Die einfachste Iteration.
- 3 **Skills zum Fragen nutzen** — Nach dieser Session kannst du ihn direkt für dich einen Skill erstellen lassen um das ganze immer wiederholen zu können.

"Entwickler sind wichtiger denn je, niemand wird einfach ersetzt. Man muss mit der Zeit gehen, dass war in diesem Feld aber immer so."

"In Zukunft entwickeln wir Agenten statt Applikationen."

Danke fürs Zuhören, Fragen?



Fabian Meyer
Chapter Lead **Vue** @ eventim Tech

Rheinwerk Spotlight "Coding mit KI" 2026



github.com/dinooo13